REȚELE DE CALCULATOARE · SEMESTRUL 1 · CURSUL 2

Arhitectură și performanță: niveluri, modele de referință și ce este în spatele întârzierii

Pachetul didactic al săptămânii 2: note de curs, seminar, laborator Linux și laborator Windows (câte două ore fiecare)

Conf.univ.dr. Popescu-Bodorin Nicolaie, 2026
+40722882471, bodorin(on)live.com, https://lmrec.org/bodorin/

Convenție de terminologie

La prima apariție în fiecare parte a documentului, termenii de specialitate traduși în română sunt urmați, în paranteză, de termenul consacrat în limba engleză sau de forma „romgleză” folosită curent de practicieni — de exemplu, **comutator de pachete** (*packet switch*), **memorie tampon** (*buffer*, în romgleză „buffer”). Comenzile, numele câmpurilor din Wireshark și rezultatele afișate de programe sunt lăsate în engleză, așa cum apar pe ecran.

Partea	Conținut	Durată
A	Cursul 2 — Arhitectură și performanță: niveluri, modele de referință și cifrele din spatele întârzierii	2 h
B	Seminarul 2 — Antete, cozi și debit	2 h
C	Laboratorul 2 (Linux) — Analiza pachetelor cu Wireshark și tcpdump	2 h
D	Laboratorul 2 (Windows) — Analiza pachetelor cu Wireshark și pktmon	2 h

Partea A · Cursul 2

După acest curs ar trebui să puteți

- să deosebiți serviciul, interfața și protocolul unui nivel și să explicați comunicarea între entități pereche;
- să urmăriți un mesaj prin încapsulare și demultiplexare, numind câmpul de antet folosit la fiecare pas;
- să descrieți cele șapte niveluri OSI, să le comparați cu modelul TCP/IP și să explicați de ce IP este „talia îngustă”;
- să spuneți cine elaborează standardele Internetului și cum devine un RFC standard;
- să folosiți intensitatea traficului pentru a prevedea întârzierea în coadă și să explicați de ce întârzierea explodează aproape de încărcarea maximă;
- să calculați debitul capăt-la-capăt și produsul lățime de bandă–întârziere al unui traseu;
- să enunțați argumentul capăt-la-capăt și să îl aplicați unei probleme de proiectare.

Planul celor două ore

Partea	Subiect	Timp
Ora 1		
1	Recapitularea Cursului 1 și o întrebare de început	5 min
2	De ce niveluri: servicii, interfețe, protocoale	15 min
3	Încapsularea și demultiplexarea, în detaliu	15 min
4	Modele de referință: OSI și TCP/IP; talia îngustă	15 min
	Pauză	10 min
Ora 2		
5	Cine stabilește regulile: standarde și RFC-uri	8 min
6	Întârzierea în coadă și pierderile, în profunzime	17 min
7	Debitul și produsul lățime de bandă–întârziere	10 min
8	Argumentul capăt-la-capăt	10 min
9	Cum măsoară ping și traceroute toate acestea; rezumat	5 min

1. Unde am rămas

Săptămâna trecută am desfăcut Internetul în gazde (*hosts*), legături (*links*) și comutatoare de pachete (*packet switches*), am văzut că este o rețea de rețele operate independent, am întâlnit cele patru componente ale întârzierii (*delay*) și am văzut — într-un singur tabel — că munca este împărțită în cinci niveluri (*layers*). Astăzi privim de aproape această împărțire și cifrele. Două întrebări străbat cursul.

- **Cum este împărțită munca?** De ce niveluri, ce niveluri și ce anume trece între ele.
- **Cât de bine funcționează?** Ce determină întârzierea, pierderile (*loss*) și debitul (*throughput*) și de ce o rețea poate trece de la rapidă la inutilizabilă printr-o creștere mică a traficului.

> **Întrebare de început**
> Descărcați un fișier pe un traseu ale cărui legături au 1 Gbit/s, 10 Gbit/s și 100 Mbit/s. Traseul vecinului vostru are 1 Gbit/s peste tot, dar timpul dus-întors (*round-trip time*, RTT) este de 300 ms în loc de 10 ms. Cine termină primul o descărcare de 1 MB? Dar una de 10 GB? Păstrați-vă răspunsurile; le lămurim în secțiunea 7.

2. De ce niveluri

Un sistem mare este înțeles și construit împărțindu-l în părți cu granițe clare. Rețelele se împart **pe verticală**: fiecare nivel folosește ce îi oferă nivelul de dedesubt și adaugă ceva, astfel încât nivelul de deasupra să primească un serviciu mai bun. Trei cuvinte trebuie ținute separat.

Termen	Înțeles	Exemplu (nivelul transport)
Serviciu (*service*)	*Ce* oferă un nivel nivelului de deasupra.	Livrarea sigură și în ordine a unui flux de octeți între două procese.
Interfață (*interface*)	*Cum* cere nivelul de deasupra serviciul: operațiile și parametrii lor.	Apelurile socket `connect`, `send`, `recv`, `close`.
Protocol (*protocol*)	*Cum* implementează nivelul serviciul, schimbând mesaje cu corespondentul său de pe altă mașină.	TCP: segmente cu numere de secvență, confirmări (*acknowledgements*, „ACK-uri”), retransmisii.

Separarea este cea care face posibilă schimbarea. Un serviciu și interfața lui pot rămâne fixe zeci de ani, în timp ce protocolul de dedesubt este înlocuit. Browserele au folosit aceeași interfață socket când TCP a primit algoritmi noi de control al congestiei (*congestion control*); cele mai multe nu au trebuit modificate.

2.1 Comunicarea între entități pereche

Fiecare nivel de pe o mașină se poartă ca și cum ar vorbi direct cu **același nivel** de pe cealaltă mașină — **entitatea pereche** (*peer*). O entitate de transport din București „trimite un segment” unei entități de transport din Frankfurt, iar protocolul ei este definit integral în termenii acestei conversații. În realitate nimic nu circulă orizontal decât la bază: segmentul coboară prin niveluri, traversează mediul fizic și urcă din nou pe partea cealaltă. Conversația orizontală este **logică**; drumul vertical este **real**.

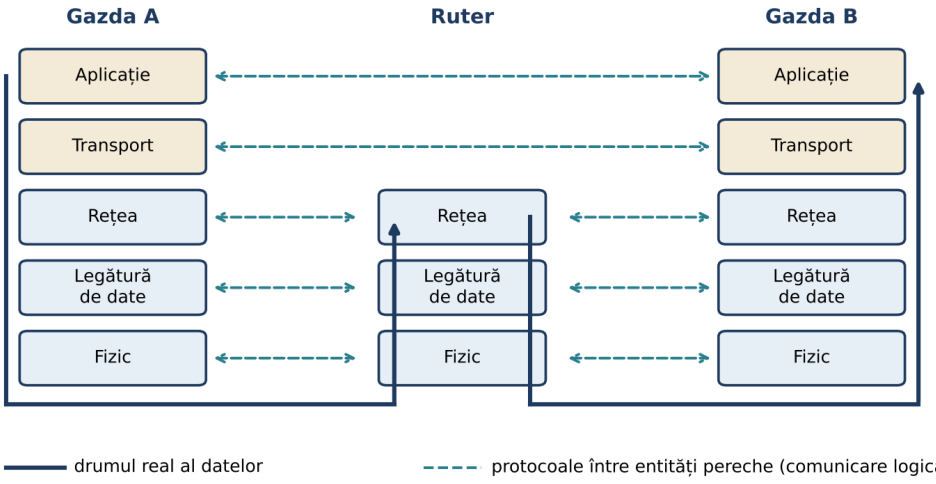


Figura 1. Protocoalele între entități pereche sunt logice; datele coboară efectiv, traversează legăturile fizice și urcă. Ruterul implementează doar cele trei niveluri de jos.

Observați în Figura 1 că protocoalele de aplicație și de transport rulează numai între cele două gazde: ruterele nu participă la ele. Protocoalele de rețea, de legătură de date și fizice rulează între vecini, salt cu salt (*hop by hop*).

2.2 Prețul organizării pe niveluri

- **Costul suplimentar** (*overhead*). Fiecare nivel adaugă un antet (*header*); mesajele mici plătesc scump (Seminarul 2 pune cifre pe acest lucru).
- **Funcții dublate.** Detectarea erorilor apare la nivelul legătură de date *și* la transport; uneori sunt necesare ambele, alteori nu (secțiunea 8).

- **Informație ascunsă.** Un nivel nu poate vedea ce ar avea uneori nevoie să știe. TCP nu poate afla dacă un pachet s-a pierdut din cauza congestiei sau a interferențelor radio, pentru că această informație se află la nivelul legătură de date.

Inginerii încalcă uneori intenționat organizarea pe niveluri — proiectare inter-niveluri (*cross-layer design*) —, de exemplu pentru a lăsa o aplicație video să se adapteze la condițiile radio ale unui telefon. Regula implicită rămâne însă respectarea granițelor.

3. Încapsularea și demultiplexarea

La emițător, fiecare nivel ia unitatea primită de sus, o tratează ca **sarcină utilă** opacă (*payload*; în limbajul OSI, *service data unit*, SDU) și îi adaugă propriul antet, formând propria **unitate de date de protocol** (*protocol data unit*, PDU): un segment, apoi o datagramă, apoi un cadru (*frame*). La receptor are loc operația inversă, iar la fiecare pas nivelul care primește trebuie să hotărască **cărei entități de deasupra** îi dă sarcina utilă. Această decizie este **demultiplexarea** (*demultiplexing*), iar fiecare antet are un câmp exact pentru acest scop.

Nivelul care primește unitatea	Câmpul citit pentru a alege nivelul următor	Valori tipice
Legătură de date (Ethernet)	**EtherType** (2 octeți)	0x0800 = IPv4, 0x86DD = IPv6, 0x0806 = ARP
Rețea (IPv4 / IPv6)	**Protocol** (IPv4) / **Next Header** (IPv6)	1 = ICMP, 6 = TCP, 17 = UDP
Transport (TCP / UDP)	**Portul destinație** (*destination port*, 2 octeți)	53 = DNS, 80 = HTTP, 443 = HTTPS

Porturile 0–1023 sunt **porturile bine-cunoscute** (*well-known ports*) ale serviciilor standard, 1024–49151 sunt **înregistrate** (*registered*) pentru anumite aplicații, iar 49152–65535 sunt **porturi dinamice** (*dynamic* sau *ephemeral ports*), atribuite de sistemul de operare clienților. Toate aceste numere — cu excepția valorilor EtherType, care aparțin IEEE — sunt ținute în registre publice de IANA (*Internet Assigned Numbers Authority*).

3.1 O cerere web, antet cu antet

Un browser trimite unui server web o cerere HTTP mică, de 400 de octeți. Iată ce iese din placa de rețea:

Componentă	Mărime (octeți)	Adăugată de	Conține, între altele
Antet Ethernet	14	nivelul legătură de date	adresa MAC destinație, adresa MAC sursă, EtherType 0x0800
Antet IPv4	20	nivelul rețea	adresele IP sursă și destinație, TTL, Protocol = 6, suma de control a antetului (*header checksum*)
Antet TCP	20 (+ opțiuni)	nivelul transport	portul sursă (de ex. 51234), portul destinație 80, numerele de secvență și de confirmare
Cererea HTTP	400	aplicația	`GET /index.html HTTP/1.1` și liniile de antet
Sfârșit de cadru Ethernet (*trailer*)	4	nivelul legătură de date	secvența de verificare a cadrului (*frame check sequence*, FCS; CRC-32)

Cadrul are 458 de octeți; 58 dintre ei sunt antete. Încă un fapt explică un număr pe care îl veți vedea peste tot: Ethernet transportă cel mult **1500 de octeți** de sarcină utilă pe cadru — **unitatea maximă de transmisie** (*maximum transmission unit*, **MTU**). De aceea un segment TCP peste Ethernet transportă de regulă cel mult 1500 − 20 − 20 = 1460 de octeți de date.

> **Încotro arată adresa MAC destinație**
> Adresa *IP* destinație din pachet este a serverului web, pe tot parcursul. Adresa *MAC* destinație este doar **următorul salt pe această legătură** — de obicei poarta implicită (*default gateway*). Fiecare ruter înlătură vechiul cadru și construiește unul nou, cu adrese MAC noi, în timp ce antetul IP călătorește capăt-la-capăt (cu TTL-ul scăzut cu unu). Veți verifica singuri acest lucru în Laboratorul 2.

4. Modele de referință

4.1 Modelul OSI

La sfârșitul anilor 1970, Organizația Internațională de Standardizare (ISO) și-a propus să definească o arhitectură completă, independentă de producători, pentru „interconectarea sistemelor deschise” (*Open Systems Interconnection*). **Modelul de referință OSI** (*OSI reference model*; ISO 7498, 1984) are șapte niveluri:

#	Nivelul OSI	Funcția	Unde se află în Internet
7	**Aplicație** (*application*)	Servicii folosite direct de aplicații.	protocoalele de aplicație (HTTP, SMTP, DNS)
6	**Prezentare** (*presentation*)	Reprezentarea datelor: codificare, compresie, criptare.	în aplicații și biblioteci (UTF-8, JSON, TLS)
5	**Sesiune** (*session*)	Controlul dialogului, sincronizare, puncte de reluare.	în aplicații (sesiuni HTTP, reluarea sesiunilor TLS)
4	**Transport** (*transport*)	Livrare capăt-la-capăt între procese.	TCP, UDP, QUIC
3	**Rețea** (*network*)	Rutare și retransmitere (*routing*, *forwarding*) între rețele.	IP
2	**Legătură de date** (*data link*)	Livrarea cadrelor între vecini; detectarea erorilor.	Ethernet, Wi-Fi
1	**Fizic** (*physical*)	Transmisia biților bruți printr-un mediu.	semnalizare pe cupru, fibră, radio

Un mnemonic englezesc pentru ordinea de la nivelul 1 în sus: „**P**lease **D**o **N**ot **T**hrow **S**ausage **P**izza **A**way”.

4.2 Modelul TCP/IP

Arhitectura proprie a Internetului nu a fost desenată niciodată atât de formal. RFC 1122 (1989) descrie patru niveluri: **aplicație, transport, internet și legătură** (*application, transport, internet, link*). Cele mai multe manuale, inclusiv al nostru, împart nivelul legătură în legătură de date și fizic și folosesc astfel **modelul cu cinci niveluri** din Cursul 1. Comparația:

OSI (7)	Modelul cu cinci niveluri (manuale)	TCP/IP, RFC 1122 (4)
Aplicație · Prezentare · Sesiune	Aplicație	Aplicație
Transport	Transport	Transport
Rețea	Rețea	Internet
Legătură de date	Legătură de date	Legătură
Fizic	Fizic	Legătură

De ce a devenit TCP/IP, și nu OSI, rețeaua lumii? Protocoalele TCP/IP existau și rulau pe mașini reale — erau livrate gratuit cu Berkeley UNIX la începutul anilor 1980 —, în timp ce protocoalele OSI au venit târziu, erau complexe și scumpe de implementat și au fost proiectate în comitete înainte de a fi folosite. **Modelul** OSI a supraviețuit ca vocabular: inginerii încă spun „switch de nivel 2” (*layer-2 switch*), „ruter de nivel 3”, „echilibrator de încărcare de nivel 7” (*layer-7 load balancer*). **Protocoalele** OSI nu au supraviețuit.

4.3 Talia îngustă

Desenați toate protocoalele Internetului pe niveluri și imaginea seamănă cu o **clepsidră** (*hourglass*; Figura 2). Sus sunt multe aplicații și protocoale de aplicație, jos multe tehnologii de legătură și medii, dar la mijloc se află, în esență, un **singur** protocol: IP. Tot ce este deasupra rulează peste IP, iar IP rulează peste tot ce este dedesubt — de unde și denumirea de **talie îngustă** (*narrow waist*).

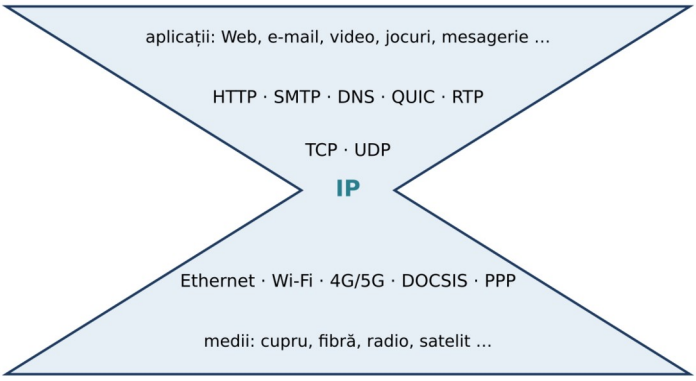


Figura 2. Clepsidra: un singur protocol de nivel rețea între multe aplicații și multe tehnologii de legătură.

Talia îngustă este cea care a făcut Internetul să crească. O aplicație nouă trebuie doar să ruleze peste IP pentru a ajunge în orice rețea; o tehnologie nouă de legătură trebuie doar să transporte IP pentru a servi orice aplicație. Costul este că talia este foarte greu de schimbat: trecerea de la IPv4 la IPv6 este în desfășurare de peste douăzeci și cinci de ani.

5. Cine stabilește regulile

Organism	Răspunde de
IETF — *Internet Engineering Task Force*	Protocoalele Internetului: IP, TCP, UDP, QUIC, DNS, HTTP, TLS și multe altele. Deschis oricui; lucrul se desfășoară public, pe liste de discuții și în grupuri de lucru (*working groups*).

Organism	Răspunde de
Comitetele **IEEE 802**	Rețele locale și metropolitane: 802.3 Ethernet, 802.11 Wi-Fi, 802.1 punți (*bridging*) și VLAN-uri.
ITU-T și **3GPP**	Standarde de telecomunicații; 3GPP specifică sistemele mobile 4G și 5G.
W3C și **WHATWG**	Tehnologiile Web: HTML, CSS, interfețele de programare ale browserelor.
IANA și **RIR**-urile	Registrele de numere și nume. IANA alocă blocuri de adrese celor cinci registre regionale de Internet (*Regional Internet Registries*, RIR); România este deservită de **RIPE NCC**, cu sediul la Amsterdam.

Un protocol Internet pornește ca **proiect Internet** (*Internet-Draft*), pe care îl poate scrie oricine. Dacă un grup de lucru îl adoptă și ajunge la un „consens aproximativ" (*rough consensus*), este publicat ca **RFC** (*Request for Comments*). Pe calea standardelor (*standards track*), un RFC este mai întâi **standard propus** (*Proposed Standard*) și poate deveni ulterior **standard Internet** (*Internet Standard*); RFC 6410 a redus în 2011 nivelurile de la trei la aceste două. RFC-urile nu se modifică după publicare; o corectură este un RFC nou, care îl *actualizează* (*updates*) sau îl *înlocuiește* (*obsoletes*) pe cel vechi. Nu toate RFC-urile sunt standarde: multe sunt informative, experimentale sau istorice — iar câteva, publicate la 1 aprilie, sunt glume.

Stilul de lucru al IETF a fost rezumat de David Clark în 1992: „Respingem regii, președinții și votul. Credem în consensul aproximativ și în codul care rulează" (*rough consensus and running code*). O specificație este luată în serios atunci când există implementări independente care funcționează împreună.

6. Întârzierea în coadă și pierderile, în profunzime

Dintre cele patru componente ale întârzierii, trei sunt simple: prelucrarea (*processing*) este mică, transmisia (*transmission*) este *L/R*, propagarea (*propagation*) este *d/s*. Interesantă este **întârzierea de așteptare în coadă** (*queueing delay*), pentru că depinde de traficul tuturor celorlalți și se schimbă de la un pachet la altul.

6.1 Intensitatea traficului

Considerați o legătură de ieșire cu rata *R* bit/s. Pachete de *L* biți sosesc pentru ea în medie cu *a* pachete pe secundă. Biții sosesc cu o rată medie de *La* bit/s și pot pleca cu *R* bit/s. Raportul

$$\rho = La / R$$

este **intensitatea traficului** (*traffic intensity*) sau **gradul de utilizare** (*utilisation*) al legăturii. Este cel mai util număr pentru a raționa despre cozi.

- **ρ > 1.** Biții sosesc mai repede decât pot pleca. Coada crește nelimitat și, cum memoriile tampon (*buffers*, în romgleză „buffere") sunt finite, se pierd pachete. Nicio mărime a memoriei tampon nu rezolvă situația. *Prima regulă a ingineriei traficului: proiectați astfel încât ρ să rămână sub 1.*
- **ρ ≤ 1**, cu sosiri perfect regulate (câte un pachet la fiecare *L/R* secunde): niciun pachet nu așteaptă.

- **ρ ≤ 1**, cu sosiri aleatoare, în rafale (*bursty*) — cazul real: pachetele sosesc uneori grămadă și așteaptă. Cu cât ρ este mai aproape de 1, cu atât așteptările sunt mai lungi și mai frecvente.

6.2 Cât de repede crește întârzierea?

Teoria cozilor (*queueing theory*) dă răspunsuri exacte pentru trafic idealizat. În cel mai simplu model, numit **M/M/1** (sosiri aleatoare independente, lungimi aleatoare ale pachetelor, o singură legătură), timpul mediu petrecut de un pachet în ruter — așteptare plus transmisie — este

$$d = (L / R) / (1 - \rho)$$

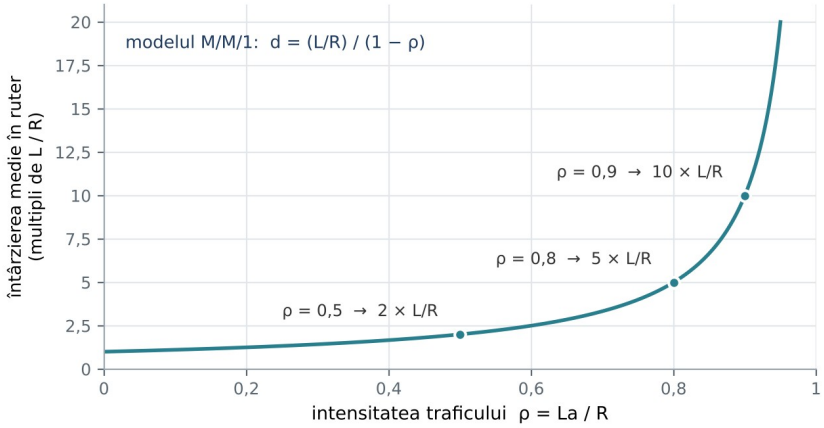


Figura 3. Întârzierea medie într-un ruter în funcție de intensitatea traficului, în modelul M/M/1.

Traficul real nu este M/M/1, așa că cifrele exacte nu trebuie luate ca atare. **Forma** curbei, însă, da. La jumătate de încărcare, un pachet petrece în medie dublul timpului său de transmisie; la 80 %, de cinci ori; la 90 %, de zece ori; la 95 %, de douăzeci de ori. Trecerea de la 90 % la 95 % utilizare — o creștere a traficului de 5 % — dublează întârzierea. De aceea operatorii de rețea modernizează legăturile cu mult înainte ca ele să fie „pline” și de aceea o rețea care mergea bine ieri poate părea stricată azi.

6.3 Pierderile și mărimea memoriilor tampon

Când coada este plină, pachetele care sosesc sunt aruncate — **aruncare la coada plină** (*tail drop*). Proporția celor pierdute crește cu ρ. Un pachet pierdut este retransmis de emițător (dacă se folosește TCP) sau pur și simplu lipsește (dacă nu), astfel că utilizatorii percep pierderile ca întârziere suplimentară sau ca sincope.

Dacă pierderile sunt rele, de ce să nu dăm ruterelor memorii tampon uriașe? Pentru că o memorie tampon mare și plină este doar o coadă lungă: fiecare pachet așteaptă în spatele ei. Memoria ieftină a făcut ca multe rutere și modemuri casnice să fie construite cu memorii tampon foarte mari, astfel încât o singură încărcare mare (*upload*) putea adăuga sute de milisecunde fiecărui pachet, inclusiv celor ale unui apel video. Problema a primit numele de **bufferbloat** („umflarea bufferelor”, în jurul anului 2010). Soluția nu este doar mărirea sau micșorarea memoriilor tampon, ci **gestionarea activă a cozilor** (*active queue management*, AQM): algoritmi precum CoDel, care aruncă sau marchează pachete din timp, pentru a menține scurtă coada permanentă. Veți experimenta cu ea în semestrul 2.

7. Debitul și produsul lățime de bandă–întârziere

7.1 Debitul capăt-la-capăt

Pentru un transfer lung pe un traseu cu legături de rate $R_1, R_2, \ldots, R_n$, debitul (*throughput*) este limitat de cea mai lentă: **debit = min($R_1, \ldots, R_n$)**. Când o legătură este **partajată** (*shared*), contează partea pe care o primește fiecare flux (*flow*). Dacă zece transferuri traversează o legătură din nucleu de rată R, iar legătura își împarte corect (*fairly*) capacitatea, fiecare primește cel mult $R/10$. O situație frecventă este ca nucleul să fie rapid și puțin încărcat, iar **legăturile de acces** (*access links*) să fie legătura îngustă (*bottleneck*); pe o legătură de nucleu congestionată, însă, capacitatea partajată poate deveni limita.

7.2 Produsul lățime de bandă–întârziere

Câți biți sunt „în zbor” (*in flight*) pe un traseu în același timp — trimiși, dar încă neconfirmați? Un emițător care transmite cu rata R pe un traseu cu timpul dus-întors RTT are, în orice moment, până la

$$\mathbf{BDP = R \times RTT}$$

biți pe drum spre destinație sau cu confirmările pe drumul de întoarcere. Acesta este **produsul lățime de bandă–întârziere** (*bandwidth–delay product*, BDP). Pentru a ține traseul plin, emițătorului trebuie să i se permită să aibă cel puțin un BDP de date neconfirmate. Exemplu: la 1 Gbit/s, cu un RTT de 30 ms (aproximativ București–Frankfurt și înapoi), BDP = 30 Mbit ≈ 3,75 MB. Un emițător limitat la 64 KB neconfirmați — cum era TCP-ul original — ar atinge doar 64 KB la 30 ms ≈ 17,5 Mbit/s pe acel traseu de 1 Gbit/s. Revenim la aceasta când studiem ferestrele TCP (*TCP windows*).

> **Răspunsul la întrebarea de început**
> Transferurile mici sunt dominate de **întârziere**: o descărcare de 1 MB are nevoie de câteva drumuri dus-întors ca să stabilească conexiunea și să își mărească rata, iar fiecare drum dus-întors vă costă 10 ms, iar pe vecinul vostru 300 ms. Terminați voi primii.
> Transferurile mari sunt dominate de **debit**: legătura voastră îngustă are 100 Mbit/s, deci 10 GB durează cel puțin 800 s; traseul vecinului poate livra 1 Gbit/s, deci — dacă fereastra emițătorului acoperă BDP-ul mare de 1 Gbit/s × 300 ms = 37,5 MB — transferul durează circa 80 s. Vecinul câștigă de zece ori.
> Care legătură este „mai rapidă” depinde de mărimea a ceea ce trimiteți.

8. Argumentul capăt-la-capăt

Unde trebuie plasată o funcție într-un sistem pe niveluri — în rețea sau în gazdele de la capete? În 1984, Jerome Saltzer, David Reed și David Clark au dat un răspuns care a modelat Internetul, într-un articol intitulat „End-to-End Arguments in System Design”.

> **Argumentul capăt-la-capăt (end-to-end argument), parafrazat**
> O funcție poate fi implementată complet și corect numai cu cunoștințele și cu ajutorul aplicației de la capete. Plasarea ei în interiorul rețelei nu este, de aceea, niciodată *suficientă*. Poate fi totuși utilă ca **îmbunătățire a performanței**, dar nu ca înlocuitor al mecanismului capăt-la-capăt.

Exemplul lor este **transferul atent de fișiere** (*careful file transfer*). Să presupunem că fiecare legătură își verifică și își retransmite cadrele, astfel încât fiecare salt este „sigur”. Poate gazda receptoare să renunțe atunci la verificarea fișierului? Nu: fișierul poate fi corupt în memoria unui ruter, de o eroare dintr-un driver de disc sau de o cădere la jumătatea transferului. Doar o verificare capăt-la-capăt — o sumă de control (*checksum*) calculată de emițător peste fișier și verificată de receptor — le prinde pe toate. Iar odată ce verificarea capăt-la-capăt există, fiabilitatea perfectă salt cu salt nu mai este necesară pentru corectitudine.

Mecanismele salt cu salt merită totuși păstrate acolo unde îmbunătățesc performanța. Wi-Fi își retransmite singur cadrele pierdute, în microsecunde, pentru că pierderile radio sunt frecvente, iar lăsarea recuperării lor în seama TCP, capăt-la-capăt, ar fi mult mai lentă. Argumentul nu interzice rețelei să ajute; spune că nu ne putem **baza** pe ea.

Consecințele se văd peste tot în proiectarea Internetului: un nucleu IP simplu, fără garanții (*best effort*); fiabilitatea, ordonarea și controlul congestiei în TCP, la gazde; criptarea (TLS) capăt-la-capăt, nu legătură cu legătură. Echipamentele care încalcă principiul — **echipamentele intermediare** (*middleboxes*): dispozitive NAT, paravane de protecție (*firewalls*), proxy-uri transparente (*transparent proxies*) care inspectează sau modifică traficul — funcționează, dar fac rețeaua mai greu de schimbat. Un protocol de transport nou, pe care aceste echipamente nu îl recunosc, este adesea blocat. Acesta este unul dintre motivele pentru care QUIC rulează peste UDP și își criptează aproape toate antetele.

9. Cum măsoară instrumentele întârzierea

`ping` trimite o **cerere de ecou** ICMP (*Echo Request*, tipul 8), iar ținta răspunde cu un **răspuns de ecou** (*Echo Reply*, tipul 0). Timpul dintre ele este un RTT, care include transmisia, propagarea, așteptarea în coadă și prelucrarea, în ambele sensuri.

`traceroute` exploatează câmpul **TTL** (*time to live*, „timp de viață”) din antetul IP. Fiecare ruter îl scade cu unu și aruncă pachetul al cărui TTL ajunge la zero, trimițând înapoi un mesaj ICMP **timp depășit** (*Time Exceeded*, tipul 11). Traceroute trimite sonde (*probes*) cu TTL = 1, 2, 3, …; sonda cu TTL = *k* expiră la al *k*-lea ruter, care își dezvăluie astfel adresa. Traceroute din Linux folosește sonde UDP către porturi înalte, improbabile (de la 33434 în sus), și recunoaște destinația după răspunsul ICMP **port inaccesibil** (*Port Unreachable*); tracert din Windows folosește cereri de ecou ICMP și recunoaște destinația după răspunsul de ecou. Van Jacobson a scris primul traceroute în 1987 — un instrument construit în întregime dintr-un efect secundar al unui câmp de antet gândit pentru altceva.

Două precauții. Drumul de întoarcere al fiecărui răspuns poate diferi de drumul de dus, astfel încât un RTT nu spune unde se află întârzierea. Iar ruterele generează răspunsurile ICMP încet și cu prioritate mică, motiv pentru care un salt intermediar poate părea mai lent decât destinația.

Rezumat

- Un nivel oferă un **serviciu** (*service*) printr-o **interfață** (*interface*) și îl implementează cu un **protocol** rulat între **entități pereche** (*peers*). Comunicarea între entități pereche este logică; datele coboară, traversează și urcă.

- Fiecare nivel adaugă un antet (**încapsulare**, *encapsulation*) și folosește un câmp de antet pentru a alege receptorul de deasupra (**demultiplexare**, *demultiplexing*): EtherType, Protocol IP, portul destinație.
- Modelul **OSI** are șapte niveluri; modelul **TCP/IP**, patru (sau cinci, în manuale). OSI supraviețuiește ca vocabular; TCP/IP, ca sistem în funcțiune. IP este **talia îngustă** (*narrow waist*).
- Standardele Internetului sunt scrise de **IETF** ca **RFC**-uri; numerele sunt înregistrate de **IANA**; adresele României vin prin **RIPE NCC**.
- **Intensitatea traficului** (*traffic intensity*) ρ = La/R guvernează cozile. Peste 1, cozile cresc nelimitat; sub 1, întârzierea crește abrupt pe măsură ce ρ se apropie de 1. Memoriile tampon (*buffers*) foarte mari provoacă **bufferbloat**.
- Debitul este stabilit de **legătura îngustă** (*bottleneck*), inclusiv de partea primită pe legăturile partajate. **Produsul lățime de bandă–întârziere** (*bandwidth–delay product*) R × RTT este cantitatea de date care trebuie să fie în zbor ca să umple un traseu.
- **Argumentul capăt-la-capăt** (*end-to-end argument*): funcțiile care au nevoie de cunoștințele capetelor aparțin capetelor; rețeaua poate ajuta doar pentru performanță.

Termeni-cheie

serviciu (*service*) · interfață (*interface*) · protocol · entitate pereche (*peer*) · unitate de date de protocol (*PDU*) · sarcină utilă (*payload*) · încapsulare (*encapsulation*) · demultiplexare (*demultiplexing*) · EtherType · număr de protocol (*protocol number*) · port bine-cunoscut / înregistrat / dinamic (*well-known / registered / dynamic port*) · MTU · model OSI · nivel prezentare (*presentation layer*) · nivel sesiune (*session layer*) · model TCP/IP · talie îngustă (*narrow waist*) · IETF · proiect Internet (*Internet-Draft*) · RFC · standard propus (*Proposed Standard*) · standard Internet (*Internet Standard*) · IANA · RIR · RIPE NCC · intensitatea traficului (*traffic intensity*) · M/M/1 · aruncare la coada plină (*tail drop*) · bufferbloat · gestionare activă a cozilor (*AQM*) · legătură îngustă (*bottleneck*) · produs lățime de bandă–întârziere (*BDP*) · argumentul capăt-la-capăt (*end-to-end argument*) · ecou ICMP (*ICMP Echo*) · TTL · timp depășit (*Time Exceeded*)

Întrebări recapitulative

1. Dați un exemplu, altul decât nivelul transport, de serviciu, interfață și protocol de la același nivel.
2. În Figura 1, la ce protocoale participă ruterul? Pe care nu le vede deloc?
3. O gazdă primește un cadru. Enumerați, în ordine, cele trei câmpuri de antet pe care le citește pentru a livra sarcina utilă unui browser web.
4. De ce se schimbă adresa MAC destinație la fiecare salt, iar adresa IP destinație nu?
5. Ce niveluri OSI nu au un corespondent separat în modelul TCP/IP și unde au ajuns funcțiile lor?
6. De ce o „talie îngustă” face ușoară adăugarea de aplicații noi, dar grea schimbarea lui IP însuși?
7. O legătură este folosită în medie 60 %. Operatorul ei spune: „Mai avem 40 % capacitate liberă, nu e nevoie de modernizare.” Ce i-ați răspunde?
8. De ce o memorie tampon mai mare nu este un leac pentru pierderi?
9. Enunțați argumentul capăt-la-capăt într-o propoziție și dați un exemplu de funcție pe care rețeaua o îndeplinește totuși, pentru performanță.

Exerciții

1. O aplicație trimite mesaje de 20 de octeți peste UDP (antet de 8 octeți) și IPv6 (antet de 40 de octeți), pe Ethernet (18 octeți de antet și sfârșit de cadru). Ce fracțiune din fiecare cadru o reprezintă datele aplicației?
2. Pachete de 1000 de octeți sosesc la un ruter a cărui legătură de ieșire are 8 Mbit/s. La ce rată a sosirilor este ρ = 1? Folosind formula M/M/1, care este întârzierea medie în ruter pentru 500, 800 și 950 de pachete pe secundă?
3. Un traseu are legături de 100 Mbit/s, 40 Mbit/s și 1 Gbit/s. Legătura de 1 Gbit/s este partajată corect de 50 de transferuri lungi simultane. Ce debit obține unul dintre ele?
4. Care este produsul lățime de bandă–întârziere al unui traseu transatlantic de 10 Gbit/s, cu RTT de 80 ms, în megaocteți?
5. Într-un traceroute cu trei sonde pe salt către o destinație aflată la 14 salturi, câte sonde sunt trimise în total (presupunând că fiecare primește răspuns)? Ce salturi răspund cu timp depășit și care cu alt mesaj?

Răspunsuri

1. Cadru = 20 + 8 + 40 + 18 = 86 de octeți; fracțiunea de date 20 / 86 ≈ 23 %.
2. L/R = 8000 / 8 × 10^6 s = 1 ms, deci ρ = 1 la 1000 de pachete/s. ρ = 0,5, 0,8, 0,95 dau d = 1 / (1 − ρ) ms = 2 ms, 5 ms și 20 ms.
3. Legătura partajată oferă 1000 / 50 = 20 Mbit/s pe transfer, mai puțin decât 40 și 100 Mbit/s, deci debitul ≈ 20 Mbit/s.
4. 10^{10} × 0,08 = 8 × 10^8 biți = 100 MB.
5. 14 × 3 = 42 de sonde. Salturile 1–13 trimit timp depășit (*Time Exceeded*); destinația (saltul 14) trimite port inaccesibil (*Port Unreachable*; traceroute din Linux) sau răspuns de ecou (*Echo Reply*; tracert din Windows).

Bibliografie și pașii următori

Obligatoriu

- Kurose și Ross, *Computer Networking: A Top-Down Approach*, ediția a 8-a, capitolul 1: secțiunile despre întârziere, pierderi și debit și despre nivelurile de protocol și modelele lor de servicii.

Recomandat

- J. H. Saltzer, D. P. Reed, D. D. Clark, „End-to-End Arguments in System Design”, *ACM Transactions on Computer Systems* 2(4), 1984.
- Peterson și Davie, *Computer Networks: A Systems Approach*, capitolul 1 (secțiunile despre arhitectură și performanță).
- RFC 1122, „Requirements for Internet Hosts — Communication Layers”, secțiunea 1.1 (arhitectura Internetului în cuvintele IETF).
- J. Gettys, K. Nichols, „Bufferbloat: Dark Buffers in the Internet”, *Communications of the ACM*, 2012.

Laboratorul 2 (săptămâna aceasta)

Captura de pachete (*packet capture*, în romgleză „sniffing”) cu Wireshark. Veți captura propria cerere web, veți deschide fiecare antet din Figura 1, veți verifica ce adrese MAC și IP apar și veți măsura un RTT din stabilirea conexiunii TCP.

Cursul următor

Nivelul aplicație: arhitecturi de aplicație, socket-uri și serviciile de care au nevoie aplicațiile, apoi Webul și HTTP/1.1.

PARTEA B · SĂPTĂMÂNA 2

Seminarul 2: Antete, cozi și debit

Ședință de probleme de două ore (2 × 50 de minute), asociată Cursului 2

Acest seminar exersează cele trei deprinderi ale Cursului 2: citirea antetelor (*headers*), raționamentul despre cozi (*queues*) și găsirea legăturii înguste (*bottleneck*). Lucrați în perechi; câte o pereche prezintă fiecare problemă. Aveți nevoie de un calculator de buzunar.

La sfârșitul seminarului ar trebui să puteți

- să plasați o funcție de rețea la nivelul potrivit din ambele modele de referință;
- să decodificați un cadru Ethernet/IPv4/UDP dintr-o afișare hexazecimală (*hex dump*);
- să calculați costul suplimentar al protocoalelor (*protocol overhead*) și debitul util maxim al unei legături;
- să prevedeți întârzierea în coadă din intensitatea traficului și să folosiți legea lui Little (*Little's law*);
- să găsiți debitul unui flux pe un traseu cu legături partajate și produsul lățime de bandă–întârziere (*bandwidth–delay product*);
- să aplicați argumentul capăt-la-capăt (*end-to-end argument*) unei propuneri de proiectare.

Plan

Pas	Activitate	Timp
A	Problema 1: la ce nivel?	10 min
B	Problema 2: decodificarea manuală a unui cadru	25 min
C	Problema 3: cât din legătură este util?	15 min
	Pauză	10 min
D	Problemele 4–5: cozi — intensitatea traficului și legea lui Little	20 min
E	Problemele 6–7: legături înguste și produsul lățime de bandă–întârziere	15 min
F	Problema 8: argumentul capăt-la-capăt (discuție)	10 min
	Temă	(în F)

A. La ce nivel?

Problema 1

Pentru fiecare funcție, indicați nivelul din modelul cu cinci niveluri și din modelul OSI.

	Funcția
a	Alegerea legăturii de ieșire pentru un pachet adresat în altă țară
b	Convertirea unui document dintr-o codificare de caractere în alta
c	Retransmiterea unui segment a cărui confirmare (*acknowledgement*) nu a sosit
d	Reprezentarea unui 1 ca tranziție de tensiune pe un cablu

	Funcția
e	Detectarea faptului că un cadru a fost corupt pe o legătură Wi-Fi
f	Reluarea unei descărcări lungi de fișier de unde s-a oprit, după o deconectare
g	Livrarea unui segment sosit procesului potrivit de pe o gazdă
h	Hotărârea care dintre mai multe gazde de pe un cablu partajat poate transmite acum

Rezolvare

(a) rețea / rețea (3). (b) aplicație / prezentare (6). (c) transport / transport (4). (d) fizic / fizic (1). (e) legătură de date / legătură de date (2). (f) aplicație / sesiune (5) — în Internet, aplicațiile fac acest lucru, de exemplu cu cereri HTTP pe intervale (*range requests*). (g) transport / transport (4), folosind portul destinație. (h) legătură de date / legătură de date (2), în subnivelul de control al accesului la mediu (*medium access control*, MAC).

B. Decodificarea unui cadru

Problema 2

Un instrument de captură afișează următorul cadru, în hexazecimal, câte 16 octeți pe linie; prima coloană este deplasamentul (*offset*). (Sfârșitul de cadru Ethernet de 4 octeți nu apare, pentru că instrumentele de captură îl omit de obicei.)

```
0000  52 54 00 12 34 01 52 54 00 3a 7f 10 08 00 45 00
0010  00 3d 1c 46 40 00 40 11 06 d5 0a 14 03 39 0a 14
0020  00 35 c8 22 00 35 00 29 96 d2 3b 7a 01 00 00 01
0030  00 00 00 00 00 00 03 77 77 77 07 65 78 61 6d 70
0040  6c 65 03 6f 72 67 00 00 01 00 01
```

Folosiți această structură. **Ethernet**: adresa MAC destinație (6 octeți), adresa MAC sursă (6), EtherType (2). **IPv4**: versiunea și lungimea antetului (1 octet: cei 4 biți superiori = versiunea, cei 4 inferiori = lungimea în cuvinte de 32 de biți), tipul serviciului (*type of service*, 1), lungimea totală (*total length*, 2), identificare (*identification*, 2), indicatori și deplasament de fragment (*flags, fragment offset*, 2), TTL (1), protocol (1), suma de control a antetului (*header checksum*, 2), adresa sursă (4), adresa destinație (4). **UDP**: portul sursă (2), portul destinație (2), lungimea (2), suma de control (2).

1. Care sunt adresele MAC destinație și sursă? Ce tip de sarcină utilă (*payload*) transportă cadrul?
2. Care sunt versiunea IP și lungimea antetului? Care este lungimea totală a datagramei IP și este ea în acord cu mărimea cadrului?
3. Care sunt TTL-ul și protocolul de transport? Care sunt adresele IP sursă și destinație, în notație zecimală cu puncte?
4. Care sunt porturile sursă și destinație? Care parte este clientul? Ce protocol de aplicație este acesta?
5. Datele aplicației încep la deplasamentul 0x2a. Fără a cunoaște în detaliu formatul DNS, ce nume este căutat? (Indicație: 0x77 este codul ASCII al lui „w”, iar octeții 03, 07, 03 sunt contoare de lungime.)
6. Este adresa MAC destinație adresa MAC a mașinii 10.20.0.53? Cum ați putea afla?

Rezolvare

1. Destinație 52:54:00:12:34:01, sursă 52:54:00:3a:7f:10. EtherType 0x0800: o datagramă IPv4.
2. 0x45: versiunea 4, lungimea antetului 5 cuvinte = 20 de octeți. Lungimea totală 0x003d = 61 de octeți. Cadrul afișat are 75 de octeți = 14 (Ethernet) + 61. Este în acord.
3. TTL 0x40 = 64 (nu a traversat încă niciun ruter — 64 este o valoare inițială frecventă). Protocol 0x11 = 17 = UDP. Sursa 0a.14.03.39 = 10.20.3.57; destinația 0a.14.00.35 = 10.20.0.53.
4. Portul sursă 0xc822 = 51234, portul destinație 0x0035 = 53. Emițătorul folosește un port dinamic, iar receptorul portul bine-cunoscut DNS, deci emițătorul este clientul, care întreabă un server DNS. Lungimea UDP 0x0029 = 41 = 8 + 33 de octeți de date DNS.
5. După cei 12 octeți ai antetului DNS urmează 03 „www” 07 „example” 03 „org” 00: numele www.example.org. Ultimii octeți 00 01 00 01 înseamnă: tipul A (o adresă IPv4), clasa IN (Internet).
6. Doar dacă serverul DNS se află pe aceeași legătură cu clientul. 10.20.3.57 și 10.20.0.53 diferă în al treilea octet; cu un prefix /24 sunt în subrețele (*subnets*) diferite, așa că cadrul merge la poarta implicită (*default gateway*), iar adresa MAC este a acesteia. Verificați cu `ip neigh` / `arp -a` — exact ce faceți în Laboratorul 2.

C. Cât din legătură este util?

Problema 3

Pe fir, fiecare cadru Ethernet este precedat și de un **preambul** (*preamble*) de 8 octeți și urmat de un **interval între cadre** (*inter-frame gap*) de 12 octeți, în care nu se poate transmite nimic altceva. Un cadru transportă cel mult 1500 de octeți de sarcină utilă și 18 octeți de antet și sfârșit de cadru (*trailer*).

1. Câte durate-octet ocupă pe fir un cadru de mărime maximă?
2. Un transfer TCP lung peste IPv4 umple fiecare cadru (antet IPv4 de 20 de octeți, antet TCP de 20 de octeți). Care este rata maximă a datelor aplicației pe un Ethernet de 100 Mbit/s?
3. Repetați pentru IPv6 (antet de 40 de octeți).
4. O aplicație de voce trimite eșantioane audio de 20 de octeți peste UDP/IPv4, câte unul pe cadru. Ce fracțiune din timpul legăturii transportă audio? (Ethernet completează — *padding* — sarcinile utile mai scurte de 46 de octeți până la 46.)

Rezolvare

1. 8 + 14 + 1500 + 4 + 12 = 1538 de durate-octet.
2. Date pe cadru = 1500 − 20 − 20 = 1460 de octeți; 1460 / 1538 × 100 ≈ 94,9 Mbit/s. De aceea testele de viteză (*speed tests*) pe legături „de 100 Mbit/s” nu arată niciodată 100.
3. 1440 / 1538 × 100 ≈ 93,6 Mbit/s.
4. Sarcina utilă = 20 + 8 + 20 = 48 de octeți (peste 46, deci fără completare). Timpul pe fir = 8 + 14 + 48 + 4 + 12 = 86 de durate-octet; fracțiunea audio 20 / 86 ≈ 23 %. De aceea codecurile reale de voce trimit eșantioane mai mari, mai rar, cu prețul unei mici întârzieri suplimentare.

D. Cozi

Problema 4 — intensitatea traficului (traffic intensity)

Legătura de ieșire a unui ruter are 10 Mbit/s; pachetele au 1250 de octeți. Presupuneți un comportament M/M/1.

1. Cât este L/R? La ce rată a sosirilor (pachete/s) ajunge ρ la 1?
2. Completați tabelul pentru rate ale sosirilor de 500, 800, 900, 950 și 990 de pachete/s: ρ și întârzierea medie în ruter.
3. Traficul crește cu 5 % pe lună. Legătura este astăzi la 80 %. În câte luni se dublează întârzierea medie? Când va fi de zece ori valoarea de azi?

> **Rezolvare**
> 1. L/R = 10 000 / 10^7 s = 1 ms. ρ = 1 la 1000 de pachete/s.
> 2. ρ = 0,5, 0,8, 0,9, 0,95, 0,99 → d = 2, 5, 10, 20, 100 ms.
> 3. Azi d = 5 ms. Dublarea la 10 ms cere ρ = 0,9, adică o creștere cu factorul 0,9/0,8 = 1,125: $1{,}05^n$ = 1,125 dă n ≈ 2,4, deci în mai puțin de trei luni. De zece ori (50 ms) cere ρ = 0,98: 0,98/0,8 = 1,225, n ≈ 4,2 luni. Operatorul care așteaptă să fie „plin” așteaptă prea mult.

Problema 5 — legea lui Little (Little’s law)

Legea lui Little spune că, în orice sistem stabil, numărul mediu de elemente din interior este egal cu rata sosirilor înmulțită cu timpul mediu petrecut de fiecare înăuntru: **N = λ × T**. Este valabilă pentru orice model de sosiri.

1. Memoria tampon (*buffer*) a unui ruter conține în medie 40 de pachete; pachetele sosesc cu 2000 pe secundă. Cât timp petrece în medie un pachet în memoria tampon?
2. Un server web tratează 300 de cereri pe secundă, fiecare durând în medie 50 ms. Câte cereri sunt în curs de prelucrare, în medie, în orice moment?
3. O conexiune TCP ține 64 KB de date neconfirmate, iar RTT este de 20 ms. Care este debitul ei? (Indicație: date în zbor = rată × timp în zbor.)

> **Rezolvare**
> 1. T = N / λ = 40 / 2000 s = 20 ms.
> 2. N = 300 × 0,05 = 15 cereri.
> 3. Rata = N / T = 65 536 de octeți / 0,02 s ≈ 3,3 MB/s ≈ 26 Mbit/s. Același raționament stă în spatele produsului lățime de bandă–întârziere.

E. Legături înguste

Problema 6 — nucleu partajat

Zece servere trimit fiecare un fișier lung către zece clienți diferiți. Fiecare server este conectat printr-o legătură de acces (*access link*) de 20 Mbit/s, fiecare client printr-o legătură de acces de 8 Mbit/s, iar toate cele zece transferuri traversează o singură legătură din nucleu de rată R, partajată corect.

1. Care este debitul fiecărui transfer dacă R = 1 Gbit/s?
2. Dar dacă R = 50 Mbit/s?
3. Pentru ce valori ale lui R este legătura din nucleu legătura îngustă?

Rezolvare

1. min(20, 8, 100) = 8 Mbit/s: limitează legătura de acces a clientului. 2. min(20, 8, 5) = 5 Mbit/s: limitează legătura partajată din nucleu. 3. Când R / 10 < 8 Mbit/s, adică R < 80 Mbit/s.

Problema 7 — produsul lățime de bandă–întârziere

1. Calculați BDP pentru: (a) un LAN de campus, 1 Gbit/s, RTT 0,5 ms; (b) București–Frankfurt, 1 Gbit/s, RTT 30 ms; (c) o legătură prin satelit geostaționar, 50 Mbit/s, RTT 600 ms.
2. Pe traseul (b), un emițător poate avea cel mult 64 KB neconfirmați. Ce debit poate atinge? Ce fracțiune din legătură reprezintă?
3. Cât de mare trebuie să fie cantitatea permisă pentru a umple traseul (b)?

Rezolvare

1. (a) $10^9 \times 0{,}0005 = 0{,}5$ Mbit = 62,5 KB. (b) 30 Mbit = 3,75 MB. (c) 30 Mbit = 3,75 MB — o legătură prin satelit lentă ține în zbor tot atâtea date cât una terestră rapidă.
2. 65 535 × 8 / 0,03 ≈ 17,5 Mbit/s, sub 2 % din legătură.
3. Cel puțin un BDP: 3,75 MB. Antetul TCP original putea exprima cel mult 64 KB; opțiunea de scalare a ferestrei (*window scale*, RFC 7323) a fost adăugată pentru a elimina această limită.

F. Argumentul capăt-la-capăt

Problema 8 — discuție

Pentru fiecare propunere, spuneți dacă este în acord cu argumentul capăt-la-capăt și de ce.

1. Ruterele ar trebui să verifice suma de control (*checksum*) a fiecărui segment TCP și să le arunce pe cele corupte, ca gazdele să nu mai trebuiască să verifice.
2. Legătura radio a unui operator mobil retransmite local cadrele pierdute, de câteva ori, înainte de a renunța.
3. Pentru a economisi lățime de bandă, proxy-ul unui ISP recomprimă toate imaginile descărcate de clienții săi.
4. Aplicațiile bancare își criptează traficul capăt-la-capăt cu TLS, deși rețeaua internă a băncii este deja criptată legătură cu legătură.

Rezolvare

1. În dezacord, ca *înlocuitor*: coruperea poate avea loc și în interiorul ruterelor sau al gazdelor, ceea ce doar o verificare capăt-la-capăt prinde. Ar putea fi o optimizare, dar în practică ruterele nu verifică sumele de control TCP, din cauza costului.
2. În acord: o îmbunătățire a performanței. TCP verifică în continuare capăt-la-capăt; legătura doar recuperează mai repede pierderile radio frecvente.
3. Problematic: rețeaua modifică date pe care capetele nu i-au cerut să le modifice și nu poate ști dacă aplicația tolerează acest lucru (o imagine medicală nu tolerează). Astfel de proxy-uri au stricat aplicații și sunt astăzi ocolite de HTTPS.
4. În acord: doar capetele pot garanta confidențialitatea între ele. Criptarea pe legături protejează fiecare legătură, dar nu și datele din interiorul fiecărui echipament intermediar.

Temă

- Rezolvați exercițiile 1–5 de la sfârșitul Cursului 2 fără să vă uitați la răspunsuri.

- Decodificați manual primii 54 de octeți ai unui segment TCP SYN pe care îl capturați singuri în Laboratorul 2, la fel ca în Problema 2. Verificați fiecare câmp în Wireshark.
- Citiți secțiunea 2 din articolul Saltzer–Reed–Clark (transferul atent de fișiere) și scrieți cinci rânduri despre măsura în care argumentul se aplică livrării de e-mail.

PARTEA C · SĂPTĂMÂNA 2

Laboratorul 2 (Linux): Analiza pachetelor cu Wireshark și tcpdump

Fișă de laborator de două ore (2 × 50 de minute), asociată Cursului 2

În Cursul 2, antetele (*headers*) au fost desene. Astăzi capturați pachete reale și le deschideți. La final veți fi urmărit o cerere web prin toate nivelurile și veți fi măsurat un timp dus-întors (*round-trip time*, RTT) din interiorul unei conexiuni TCP.

> **La sfârșitul laboratorului ar trebui să puteți**
> • să capturați trafic (*packet capture*, în romgleză „sniffing”) cu Wireshark și cu `tcpdump` și să salvați și redeschideți capturile;
> • să deosebiți filtrele de captură (*capture filters*) de filtrele de afișare (*display filters*) și să le scrieți pe amândouă;
> • să identificați, într-un pachet real, antetele Ethernet, IP și TCP/UDP și câmpurile de demultiplexare din Cursul 2;
> • să urmăriți o cerere web completă: căutarea DNS, stabilirea conexiunii TCP (*three-way handshake*, „handshake-ul”), schimbul HTTP, închiderea conexiunii;
> • să măsurați RTT din stabilirea conexiunii și să îl comparați cu `ping`;
> • să urmăriți cum lucrează traceroute și să spuneți ce mai poate vedea un observator în traficul HTTPS criptat.

Înainte de a începe

- **Mediul de lucru.** Mașina virtuală a cursului sau o instalare Linux cu interfață grafică. (În WSL 2, folosiți acolo `tcpdump` și deschideți fișierele cu Wireshark din Windows.)
- **Instalați și permiteți captura ca utilizator obișnuit:**

```
sudo apt install -y wireshark tshark tcpdump traceroute curl dnsutils
# răspundeți „Yes” când sunteți întrebați dacă utilizatorii obișnuiți pot captura pachete
sudo usermod -aG wireshark $USER
# deconectați-vă și reconectați-vă, apoi verificați:
groups | grep wireshark
```

- **Reguli de conduită.** Capturați numai pe propria mașină și numai propriul trafic. Capturarea traficului altor persoane fără permisiune este lipsită de etică și, în multe situații, ilegală.
- **Ce predați.** Un raport cu răspunsurile la **Q1–Q16** și fișierul de captură `web.pcapng` din Sarcina 3.
- **Numele interfeței.** Mai jos, `eth0` ține locul interfeței voastre principale; aflați-o cu `ip -br link` (poate fi `enp0s3`, `ens33`, `wlp2s0`…).

Adresele și valorile din această fișă sunt ilustrații. Ale voastre vor fi diferite.

Plan

Pas	Activitate	Timp
1	O primă captură și un tur al Wireshark	15 min
2	Filtre de captură și filtre de afișare	10 min

Pas	Activitate	Timp
3	O cerere web, nivel cu nivel	25 min
	Pauză	10 min
4	Întreaga conversație: fluxuri, statistici, graful fluxului	10 min
5	Măsurarea RTT din stabilirea conexiunii	10 min
6	Traceroute la microscop	15 min
7	Captura din linia de comandă: tcpdump și tshark	10 min
8	HTTPS: ce rămâne vizibil	5 min

Sarcina 1 — O primă captură

Porniți Wireshark, faceți dublu clic pe interfața principală și, într-un terminal, rulați:

```
curl -s http://example.org/ > /dev/null
```

Opriți captura (pătratul roșu). Wireshark arată trei panouri: **lista pachetelor** (*packet list*; o linie pe pachet), **detaliile pachetului** (*packet details*; antetele pachetului selectat, câte o linie extensibilă pe nivel) și **octeții pachetului** (*packet bytes*; hexazecimalul brut). Un clic pe un câmp din detalii îi evidențiază octeții — legătura dintre tabelele din Cursul 2 și realitate.

- **Q1.** Câte pachete ați capturat în total și cam câte dintre ele aparțin comenzii `curl`? Ce mai trimite mașina voastră fără să i-o cereți?
- **Q2.** Selectați un pachet oarecare și extindeți-i detaliile. Ce niveluri enumeră Wireshark? Puneți-le în corespondență cu modelul cu cinci niveluri.

Sarcina 2 — Filtre de captură și filtre de afișare

Există două feluri de filtre, cu **sintaxă diferită**:

	Filtru de captură (*capture filter*)	Filtru de afișare (*display filter*)
Când acționează	înainte de captură: pachetele care nu corespund nu sunt înregistrate	după captură: pachetele care nu corespund sunt doar ascunse
Sintaxă	BPF, ca în tcpdump	limbajul propriu de câmpuri al Wireshark
Exemple	`host example.org` · `port 53` · `tcp port 80` · `icmp`	`dns` · `http` · `tcp.port == 80` · `ip.addr == 10.20.0.53` · `icmp` · `tcp.flags.syn == 1`

Reporniți captura cu filtrul de captură `port 53 or port 80`, repetați comanda `curl` și apoi încercați, pe rând, filtrele de afișare din tabel.

- **Q3.** Scrieți un filtru de afișare care arată doar răspunsurile DNS (indicație: câmpul `dns.flags.response`) și unul care arată doar pachetele trimise *de* mașina voastră.

Sarcina 3 — O cerere web, nivel cu nivel

Goliți memoria cache DNS (*DNS cache*), astfel încât cererea să înceapă cu o căutare DNS reală, apoi capturați tot și salvați fișierul:

```
resolvectl flush-caches        # pe sistemele cu systemd-resolved
# porniți o captură în Wireshark (fără filtru de captură), apoi:
curl -s http://example.org/ > /dev/null
# opriți, apoi File → Save As… web.pcapng
```

Aplicați filtrul de afișare `dns or tcp.port == 80`. Ar trebui să găsiți, în ordine: o interogare DNS (*query*) și răspunsul ei; trei pachete TCP cu indicatorii (*flags*) SYN, SYN-ACK, ACK — **stabilirea conexiunii** (*three-way handshake*); cererea HTTP GET; răspunsul `200 OK`; și pachete cu indicatorul FIN, care închid conexiunea.

Selectați pachetul HTTP GET și completați în raport tabelul:

Nivel	Câmp	Valoarea din captura voastră
Ethernet	MAC sursă / MAC destinație	
Ethernet	Type (EtherType)	
IPv4	Adresa sursă / destinație	
IPv4	TTL · Protocol · Total length	
TCP	Portul sursă / portul destinație	
TCP	Sequence number · Header length	
HTTP	Linia de cerere · antetul Host	

- **Q4.** Care dintre valorile din tabel sunt cheile de demultiplexare din Cursul 2? Ce s-ar întâmpla dacă EtherType ar fi 0x86DD?
- **Q5.** Comparați adresa MAC destinație a cererii GET cu adresa MAC a porții implicite (*default gateway*; `ip neigh`). Explicați rezultatul.
- **Q6.** Uitați-vă la TTL-ul *răspunsului* serverului. Estimați numărul de rutere dintre voi și server. De ce ar putea drumul de dus și cel de întoarcere să aibă lungimi diferite?
- **Q7.** Câți octeți de antete precedă primul octet HTTP în cadrul GET? Verificați cu Cursul 2, secțiunea 3.1.

Sarcina 4 — Întreaga conversație

- Clic dreapta pe pachetul GET → **Follow → TCP Stream** (urmărirea fluxului TCP). Wireshark arată datele aplicației din ambele sensuri, colorate.
- **Statistics → Protocol Hierarchy** (ierarhia protocoalelor) arată cum se împart octeții capturați între protocoale.
- **Statistics → Conversations** (conversații) enumeră fiecare pereche de capete și octeții schimbați.
- **Statistics → Flow Graph** (graful fluxului) desenează schimbul ca diagramă timp–secvență, ca diagramele de la curs.
- **Q8.** Desenați (sau lipiți) graful fluxului pentru cererea voastră web și etichetați: căutarea DNS, stabilirea conexiunii, cererea, răspunsul, închiderea. Câte pachete a costat aducerea unei singure pagini mici?

Sarcina 5 — Măsurarea RTT din stabilirea conexiunii

Setați **View → Time Display Format → Seconds Since Previous Displayed Packet** (secunde de la pachetul afișat anterior). Aplicați filtrul de afișare `tcp.flags.syn == 1`, care arată SYN-ul și SYN-ACK-ul.

- **Q9.** Timpul dintre SYN-ul vostru și SYN-ACK-ul serverului este un RTT. Cât este? Rulați acum `ping -c 5 example.org` și comparați. Explicați eventualele diferențe. (Wireshark îl calculează și singur: căutați câmpul `tcp.analysis.ack_rtt` sub *[SEQ/ACK analysis]* la pachetele ACK.)
- **Q10.** În schimbul DNS, cât i-a luat rezolvitorului (*resolver*) să răspundă? Repetați `curl` fără a goli memoria cache: mai apare o căutare DNS? De ce?

Sarcina 6 — Traceroute la microscop

Capturați cu filtrul de afișare `icmp or (udp and ip.dst == <adresa lui example.org>)` în timp ce rulați un traceroute scurt:

```
traceroute -n -q 1 -m 6 example.org
```

- **Q11.** Găsiți sondele (*probes*) trimise. Ce protocol și ce porturi destinație folosesc? Care este TTL-ul primei, celei de-a doua, celei de-a treia sonde?
- **Q12.** Găsiți răspunsurile. Ce tip și cod ICMP trimit ruterele înapoi? Deschideți unul: în interiorul mesajului ICMP, Wireshark arată începutul sondei *voastre* originale. De ce este util ca mesajul ICMP să o transporte?

Sarcina 7 — Captura din linia de comandă

Pe servere nu există interfață grafică. Aceleași capturi se fac cu `tcpdump` (și se analizează cu `tshark`, geamănul de linie de comandă al Wireshark):

```
sudo tcpdump -i eth0 -n -c 20 'port 53 or port 80'
sudo tcpdump -i eth0 -n -w cli.pcap 'host example.org' &
curl -s http://example.org/ > /dev/null; sudo pkill tcpdump
tshark -r cli.pcap
tshark -r cli.pcap -Y http.request -T fields -e ip.src -e ip.dst -e http.host -e
http.request.uri
```

- **Q13.** Deschideți `cli.pcap` în Wireshark. Este aceeași conversație ca în Sarcina 3? Care este diferența dintre prezența și absența opțiunii `-n` în rezultatul afișat de `tcpdump`?

Sarcina 8 — HTTPS: ce rămâne vizibil

Capturați în timp ce rulați `curl -s https://example.org/ > /dev/null` și aplicați filtrul de afișare `tls`.

- **Q14.** Puteți citi cererea HTTP? În primul mesaj TLS (*Client Hello*), găsiți extensia **server_name** (*Server Name Indication*, SNI). Ce conține?
- **Q15.** Enumerați ce poate afla în continuare un observator din rețeaua voastră despre această vizită HTTPS (gândiți-vă la adrese, nume, mărimi și momente de timp).
- **Q16.** Ce niveluri din Cursul 2 sunt criptate de TLS și care circulă în clar? Puneți răspunsul în legătură cu argumentul capăt-la-capăt.

Dacă ceva nu merge

Simptom	Cauza probabilă și ce să încercați
Wireshark nu afișează nicio interfață	Nu sunteți încă în grupul `wireshark`. Rulați `groups`; deconectați-vă și reconectați-vă.
Nu apare nicio interogare DNS	Răspunsul era în memoria cache. Goliți-o (`resolvectl flush-caches`) sau interogați alt nume.
Nu apare HTTP, doar TLS	Site-ul v-a redirecționat (*redirect*) spre HTTPS. Folosiți exact `http://example.org/` sau `curl` fără `-L`.
Captura este inundată de alt trafic	Folosiți un filtru de captură sau un filtru de afișare pe adresa serverului.
`tcpdump: permission denied`	Puneți `sudo` în față sau folosiți permisiunea de grup a Wireshark.

PARTEA D · SĂPTĂMÂNA 2

Laboratorul 2 (Windows): Analiza pachetelor cu Wireshark și pktmon

Fișă de laborator de două ore (2 × 50 de minute), asociată Cursului 2

Această fișă face pe Windows ce face fișa de Linux pe Linux. Wireshark este același program pe ambele sisteme; diferă modul în care sunt capturate pachetele dedesubt, instrumentele care generează traficul și instrumentul de captură din linia de comandă inclus în sistem.

La sfârșitul laboratorului ar trebui să puteți

- să capturați trafic (*packet capture*) cu Wireshark pe Windows și cu `pktmon`, inclus în sistem;
- să deosebiți filtrele de captură (*capture filters*) de filtrele de afișare (*display filters*) și să le scrieți pe amândouă;
- să identificați antetele Ethernet, IP și TCP/UDP și câmpurile de demultiplexare în pachete reale;
- să urmăriți o cerere web completă și să măsurați RTT din stabilirea conexiunii TCP (*three-way handshake*);
- să explicați, pornind de la pachetele însele, cum diferă `tracert` din Windows de `traceroute` din Linux;
- să spuneți ce mai poate vedea un observator în traficul HTTPS.

Înainte de a începe

- **Instalați Wireshark** de pe wireshark.org dacă imaginea laboratorului nu îl are. Programul de instalare instalează și **Npcap**, driverul care permite programelor Windows să captureze pachete; acceptați-l. Instalarea cere drepturi de administrator; captura ulterioară, nu.
- **Terminalul.** Folosiți PowerShell. Unele comenzi sunt marcate *(admin)*: pentru acestea, deschideți terminalul cu „Run as administrator”.
- **Notă despre Wi-Fi.** Pe un adaptor Wi-Fi, Windows îi dă Wireshark-ului cadre care *arată ca* niște cadre Ethernet: antetele 802.11 reale nu apar decât dacă adaptorul suportă modul monitor (*monitor mode*). La fel se întâmplă și în Linux, în modul obișnuit. Totul din această fișă funcționează în ambele situații.
- **Regulile de conduită și ce predați** sunt ca în fișa de Linux: răspunsurile la **Q1–Q16** și fișierul `web.pcapng` din Sarcina 3.

Adresele și valorile din această fișă sunt ilustrații. Ale voastre vor fi diferite.

Plan

Pas	Activitate	Timp
1	O primă captură și un tur al Wireshark	15 min
2	Filtre de captură și filtre de afișare	10 min
3	O cerere web, nivel cu nivel	25 min
	Pauză	10 min
4	Întreaga conversație: fluxuri, statistici, graful fluxului	10 min

Pas	Activitate	Timp
5	Măsurarea RTT din stabilirea conexiunii	10 min
6	tracert la microscop	15 min
7	Captura din linia de comandă: pktmon și tshark	10 min
8	HTTPS: ce rămâne vizibil	5 min

Sarcina 1 — O primă captură

Porniți Wireshark și faceți dublu clic pe adaptorul care transportă trafic (cel cu linia de activitate în mișcare: de obicei *Ethernet* sau *Wi-Fi*). În PowerShell, rulați:

```
curl.exe -s http://example.org/ -o NUL
```

Opriți captura. Identificați cele trei panouri — lista pachetelor (*packet list*), detaliile pachetului (*packet details*), octeții pachetului (*packet bytes*) — și faceți clic pe un câmp din detalii ca să îi vedeți octeții evidențiați.

- **Q1.** Câte pachete ați capturat și cam câte aparțin comenzii voastre? Mașinile Windows sunt „vorbărețe”: numiți alte trei protocoale pe care le vedeți (uitați-vă la coloana *Protocol*).
- **Q2.** Extindeți detaliile unui pachet oarecare. Puneți nivelurile enumerate de Wireshark în corespondență cu modelul cu cinci niveluri.

Sarcina 2 — Filtre de captură și filtre de afișare

	Filtru de captură (*capture filter*)	Filtru de afișare (*display filter*)
Când acționează	înainte de captură: pachetele care nu corespund nu sunt înregistrate	după captură: pachetele care nu corespund sunt doar ascunse
Sintaxă	BPF, ca în tcpdump	limbajul propriu de câmpuri al Wireshark
Exemple	`host example.org` · `port 53` · `tcp port 80` · `icmp`	`dns` · `http` · `tcp.port == 80` · `ip.addr == 10.20.0.53` · `icmp` · `tcp.flags.syn == 1`

Reporniți cu filtrul de captură `port 53 or port 80`, repetați `curl.exe` și încercați pe rând filtrele de afișare.

- **Q3.** Scrieți un filtru de afișare care arată doar răspunsurile DNS și unul care arată doar pachetele trimise de mașina voastră (aflați-vă adresa cu `ipconfig`).

Sarcina 3 — O cerere web, nivel cu nivel

Goliți memoria cache DNS (*DNS cache*), ca cererea să înceapă cu o căutare reală, capturați fără filtru și salvați:

```
ipconfig /flushdns
# porniți o captură în Wireshark, apoi:
curl.exe -s http://example.org/ -o NUL
# opriți, apoi File → Save As… web.pcapng
```

Cu filtrul de afișare `dns or tcp.port == 80`, găsiți: interogarea și răspunsul DNS; stabilirea conexiunii (SYN, SYN-ACK, ACK); cererea HTTP GET; răspunsul `200 OK`; pachetele FIN de închidere. Selectați GET și completați tabelul:

Nivel	Câmp	Valoarea din captura voastră
Ethernet	MAC sursă / MAC destinație	
Ethernet	Type (EtherType)	
IPv4	Adresa sursă / destinație	
IPv4	TTL · Protocol · Total length	
TCP	Portul sursă / portul destinație	
TCP	Sequence number · Header length	
HTTP	Linia de cerere · antetul Host	

- **Q4.** Care valori sunt cheile de demultiplexare din Cursul 2?
- **Q5.** Comparați adresa MAC destinație a cererii GET cu adresa MAC a porții implicite (*default gateway*; `arp -a` sau `Get-NetNeighbor`). Explicați.
- **Q6.** Ce TTL pune mașina voastră în pachetele trimise? Comparați cu TTL-ul din răspunsul serverului și estimați numărul de rutere dintre voi.
- **Q7.** Câți octeți de antete precedă primul octet HTTP? Verificați cu Cursul 2, secțiunea 3.1.

Sarcina 4 — Întreaga conversație

Folosiți **Follow → TCP Stream**, **Statistics → Protocol Hierarchy**, **Statistics → Conversations** și **Statistics → Flow Graph**, ca în fișa de Linux.

- **Q8.** Lipiți graful fluxului (*flow graph*) pentru cererea voastră și etichetați: căutarea DNS, stabilirea conexiunii, cererea, răspunsul, închiderea. Câte pachete a costat o singură pagină mică?

Sarcina 5 — Măsurarea RTT din stabilirea conexiunii

Setați **View → Time Display Format → Seconds Since Previous Displayed Packet** și aplicați `tcp.flags.syn == 1`.

- **Q9.** Cât este timpul dintre SYN și SYN-ACK? Comparați cu `ping -n 5 example.org` și cu câmpul `tcp.analysis.ack_rtt`. Explicați eventualele diferențe.
- **Q10.** Cât i-a luat serverului DNS să răspundă? Repetați `curl.exe` fără a goli memoria cache: mai apare o căutare DNS? Uitați-vă în `ipconfig /displaydns` ca să vedeți de ce.

Sarcina 6 — tracert la microscop

Capturați cu filtrul de afișare `icmp` în timp ce rulați:

```
tracert -d -h 6 -w 1000 example.org
```

- **Q11.** Ce fel de pachete trimite `tracert`? Ce TTL au, salt după salt, și câte sunt trimise pe salt? Cum diferă de `traceroute` din Linux (Cursul 2, secțiunea 9)?
- **Q12.** Ce tip ICMP trimit ruterele înapoi? Deschideți unul: ce arată Wireshark în interiorul mesajului ICMP și de ce este util?

Sarcina 7 — Captura din linia de comandă: pktmon și tshark

Windows are un monitor de pachete inclus, **pktmon**, care cere un terminal de administrator. El scrie un format propriu, pe care îl poate converti pentru Wireshark:

```
# (admin)
pktmon start --capture --comp nics --pkt-size 0 -f cli.etl
curl.exe -s http://example.org/ -o NUL
pktmon stop
pktmon etl2pcap cli.etl -o cli.pcapng
```

Wireshark instalează și `tshark`, versiunea lui de linie de comandă:

```
& "C:\Program Files\Wireshark\tshark.exe" -D
& "C:\Program Files\Wireshark\tshark.exe" -r cli.pcapng -Y http.request -T fields -e
ip.src -e ip.dst -e http.host -e http.request.uri
```

- **Q13.** Deschideți `cli.pcapng` în Wireshark. Este aceeași conversație ca în Sarcina 3? `pktmon` vede fiecare pachet în mai multe puncte din interiorul stivei de rețea (*network stack*): observați pachete care apar de mai multe ori?

Sarcina 8 — HTTPS: ce rămâne vizibil

Capturați în timp ce rulați `curl.exe -s https://example.org/ -o NUL` și aplicați filtrul de afișare `tls`.

- **Q14.** Puteți citi cererea HTTP? În *Client Hello*, găsiți extensia **server_name** (*Server Name Indication*, SNI). Ce conține?
- **Q15.** Enumerați ce poate afla în continuare un observator din rețeaua voastră despre această vizită HTTPS.
- **Q16.** Ce niveluri sunt criptate de TLS și care circulă în clar? Puneți răspunsul în legătură cu argumentul capăt-la-capăt (*end-to-end argument*).

Linux și Windows față în față

Scop	Linux	Windows
Driverul de captură	nucleul sistemului (libpcap)	Npcap
Captură grafică	Wireshark	Wireshark
Captură inclusă, din linia de comandă	`tcpdump`	`pktmon` *(admin)*
Analiză din linia de comandă	`tshark`	`tshark.exe`
Golirea memoriei cache DNS	`resolvectl flush-caches`	`ipconfig /flushdns`
Sondele traceroute	UDP către porturi înalte (implicit)	cerere de ecou ICMP (*Echo Request*)
TTL inițial tipic	64	128

Dacă ceva nu merge

Simptom	Cauza probabilă și ce să încercați
Wireshark nu afișează nicio interfață	Npcap lipsește sau nu a fost instalat. Rulați din nou programul de instalare Wireshark și selectați Npcap.

Simptom	Cauza probabilă și ce să încercați
Captura pe adaptorul ales nu arată nimic	Ați ales un adaptor virtual (VPN, Hyper-V). Alegeți-l pe cel a cărui linie de activitate se mișcă.
Nu apare nicio interogare DNS	Numele era în memoria cache. Rulați din nou `ipconfig /flushdns` chiar înainte de cerere.
`pktmon` raportează „access denied”	Deschideți terminalul cu „Run as administrator”.
`curl` afișează un obiect, nu o pagină	Ați rulat aliasul din PowerShell. Tastați `curl.exe`.

Săptămâna viitoare

Cursul 3 deschide nivelul aplicație cu Webul și HTTP. În Laboratorul 3 scrieți primele programe de rețea: un client și un server TCP folosind socket-uri.